

FENIXCMS_6

FASE 6 - PRODUCTION GATE: FUENTE ÚNICA DE VERDAD, PAYMENT ATÓMICO, FALLBACKS CERRADOS Y RECUPERACIÓN

INICIO FASE 6

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

VALIDACIÓN LOCAL:

ORDENADOR DEBIAN

DESPLIEGUE:

NO VPS PRODUCCIÓN HASTA CERRAR ESTA FASE

PRIORIDAD:

CRÍTICA - BLOQUEADOR DIRECTO PARA PRODUCCIÓN.

1. RESULTADO DE LA AUDITORÍA

La revisión del main actual confirma que FenixCMS tiene una base sólida de PostgreSQL, Prisma, aislamiento multi-tenant, PayPal server-side y protecciones del checkout de tienda, pero todavía conserva una segunda realidad de runtime en la parte comercial.

El hallazgo máximo está en:

lib/services/payment.service.ts

La función createSaaSCheckoutSession todavía:

- importa INITIAL_APPLICATIONS, INITIAL_PLANS e INITIAL_TENANTS;
- puede caer a INITIAL_* si DB no devuelve datos;
- calcula precios de fallback;
- genera tenantId antes de que exista un pago capturado;
- llama a ensureTenantExists durante el checkout;
- mantiene FALLBACK_PAYMENTS;
- devuelve URLs de checkout que no representan necesariamente una sesión real del proveedor.

Además, verifyAndProcessSaaSPayment todavía:

- permite localizar Payment desde fallback de memoria;
- toma plan/application/billingPeriod desde metadata/defaults;
- puede marcar Payment como COMPLETED en el objeto en memoria cuando falla el update PostgreSQL;
- crea Subscription, License e Invoice fuera de una única transacción comercial;
- utiliza INITIAL_PLANS para datos de licencia.

Esto significa que la Fase 5 identifica correctamente el problema, pero FENIXCMS_6 debe convertirlo en una barrera real de producción.

2. OBJETIVO MÁXIMO

En producción debe existir una sola realidad:

SaaS:

PostgreSQL + PayPal real.

Storefront:

PostgreSQL + proveedor de pago real.

Auth:

PostgreSQL + PasswordService + SessionService.

Media:

PostgreSQL + storage persistente.

Webhooks:

proveedor verificado + PostgreSQL.

Super Admin:

PostgreSQL.

La memoria y INITIAL_* solo podrán existir en tests/dev controlados y nunca deben ser alcanzables cuando NODE_ENV=production.

3. REGLAS INVIOABLES

1. NO DATABASE = NO BUSINESS SUCCESS.
2. NO PAYMENT CAPTURED = NO SAAS PROVISIONING.
3. NO VERIFIED PAYMENT = NO PAYMENT COMPLETED.
4. NO VERIFIED WEBHOOK = NO PAYMENT UPDATE.
5. NO STORAGE WRITE = NO MEDIA SUCCESS.
6. DB EMPTY != DB DOWN.
7. CLIENT INPUT != SERVER AUTHORITY.
8. RETRY != DUPLICATE PROVISIONING.
9. CRITICAL ERROR != SUCCESS.
10. MOCK/TEST DATA != PRODUCTION DATA.

4. FASE 6A - CERRAR PaymentService

Archivo:

lib/services/payment.service.ts

Eliminar del runtime de producción:

- INITIAL_APPLICATIONS.
- INITIAL_PLANS.
- INITIAL_TENANTS.
- FALLBACK_PAYMENTS.
- precios por defecto.
- moneda usada como sustitución de DB.
- tenant creado antes del pago.
- ensureTenantExists durante create checkout.
- checkout URL ficticia.
- clientSecret ficticio.
- provider verification basada únicamente en un ID que "no parece fake".

Si DB falla:

respuesta 503/error.

No cambiar a datos demo.

5. FASE 6B - CREATE CHECKOUT NO CREA TENANT

createSaaSCheckoutSession() debe limitarse a:

1. Validar Application.
2. Validar Plan.
3. Validar relación Plan -> Application.
4. Calcular amount/currency.
5. Crear CheckoutSession persistente.
6. Crear PayPal Order real.
7. Guardar paypalOrderId.
8. Devolver approvalUrl real.

NO debe crear Tenant.

La arquitectura final:

Application

-> Plan

-> CheckoutSession

-> PayPal Order

-> Approval

-> Capture

-> Validación

-> Provisioning

-> Tenant

6. FASE 6C - CHECKOUTSESSION COMO FUENTE DE VERDAD

Reutilizar la CheckoutSession de FENIXCMS_2 si ya existe.

Debe conservar:

- id;
- provider;
- status;
- applicationId;
- planId;
- billingPeriod;
- amountExpected;
- currencyExpected;
- customerName;
- customerEmail;
- tenantName;
- tenantSlug;
- billingAddress;
- paypalOrderId;
- paypalCaptureId;
- paymentId;
- tenantId;
- expiresAt;
- approvedAt;
- capturedAt;
- completedAt;
- consumedAt;
- failureCode;
- failureReason;
- timestamps.

No almacenar secretos.

7. FASE 6D - APPLICATION Y PLAN SOLO DESDE POSTGRESQL

Application inexistente:

-> 404/422.

Plan inexistente:

-> 404/422.

Application-Plan incompatible:

-> 409/422.

DB caída:

-> 503.

Prohibido:

- devolver la primera Application;
- devolver el primer Plan;
- crear ECOMMERCE automáticamente;
- usar INITIAL_APPLICATIONS;
- usar INITIAL_PLANS.

8. FASE 6E - PRECIO SERVER-SIDE

El browser no puede decidir:

- amount;
- currency;

- activationLimit;
- entitlements;
- subscription amount;
- planName.

La CheckoutSession almacena el snapshot comercial y ese snapshot manda durante el capture.

Eliminar patrones del tipo:

```
amount || valor
currency || 'EUR'
plan || 'plan_pro'
application || 'app_ecommerce'
```

En producción, ausencia de dato = error.

9. FASE 6F - PAYPAL REAL

Mantener:

lib/services/paypal-gateway.service.ts

pero exigir:

- OAuth2 real;
- Create Order real;
- approval URL real;
- Capture real;
- Order ID real;
- Capture ID real;
- status;
- amount;
- currency;
- reference.

La documentación oficial actual de PayPal indica que Orders v2 debe invocarse desde el servidor, que el comprador debe aprobar la orden y que después se captura mediante /v2/checkout/orders/ORDER-ID/capture. ■cite■turn978776search0■turn978776search2■

10. FASE 6G - APPROVAL URL Y FRONTEND

components/saas/SaaS Landing.tsx debe hacer:

```
~~~text
POST /api/billing/checkout
|
v
approvalUrl real
|
v
window.location.assign(approvalUrl)
|
v
PayPal
|
v
billing/success
|
v
POST /api/billing/capture
~~~
```

No capturar inmediatamente después de crear la Order.

No mostrar "pago completado" antes de Capture + provisioning.

PayPal documenta el uso de la URL approve para el flujo de aprobación. ■cite■turn978776search9■

11. FASE 6H - CAPTURE ÚNICO Y SERVER-SIDE

app/api/billing/capture/route.ts

Entrada pública mínima:

```
~~~json
{
  "paypalOrderId": "ORDER-ID"
}
~~~
```

El servidor obtiene de CheckoutSession:

- plan;
- application;
- amount;
- currency;
- customer;
- tenant data;
- billing period.

No aceptar como autoridad:

amount
currency
tenantId
licenseKey
subscriptionId
invoiceId
entitlements.

12. FASE 6I - VALIDACIÓN COMPLETA DE PAYPAL

Aceptar únicamente si:

1. Order ID coincide con CheckoutSession.
2. Existe Capture ID real.
3. Capture status = COMPLETED.
4. Amount capturado = amountExpected.
5. Currency capturada = currencyExpected.
6. custom_id/reference_id coincide con CheckoutSession.
7. captureId no fue usado anteriormente.

PayPal documenta que la respuesta de captura incluye el Order ID y el capture ID dentro de la información de payments/captures.

■cite■turn978776search4■

13. FASE 6J - VERIFY WITH PROVIDER DEBE SER REAL

Eliminar la lógica:

"si el ID no es fake, se considera verificado".

Para PayPal la verificación debe depender de:

- Order real;
- Capture real;
- respuesta real;
- estado real;
- valores reales.

Un string recibido desde el navegador nunca es prueba de pago.

14. FASE 6K - PAYMENT NO PUEDE CAMBIAR A COMPLETED EN MEMORIA

Actualmente existe una ruta donde:

DB update falla

-> console.warn

-> payment.status = COMPLETED

-> continúa.

Eliminarla.

Nuevo comportamiento:

DB update falla

-> no success final

-> estado recuperable

-> no duplicate provisioning

-> reconciliation posterior.

PostgreSQL es la autoridad.

15. FASE 6L - PROVISIONING ÚNICO

Crear o consolidar:

lib/services/saas-provisioning.service.ts

Método:

provisionCapturedCheckout(checkoutSessionId, captureData)

Debe ser el único punto de creación final de:

- Payment;
- User;
- Tenant;
- Membership OWNER;
- License;
- Subscription;
- Invoice;
- System Domain;
- CheckoutSession COMPLETED.

16. FASE 6M - TRANSACCIÓN ATÓMICA

Después de Capture COMPLETED:

~~~text

BEGIN TRANSACTION

1. Releer CheckoutSession
2. Validar estado
3. Validar expiry
4. Validar captureId unique
5. Validar amount/currency/reference
6. Crear Payment
7. Crear/reutilizar User
8. Crear Tenant
9. Crear Membership OWNER
10. Crear License
11. Crear Subscription
12. Crear Invoice
13. Crear System Domain
14. Marcar CheckoutSession COMPLETED
15. Guardar consumedAt

COMMIT

~~~

Si falla cualquier operación crítica:

ROLLBACK.

No devolver success=true con recursos parciales.

17. FASE 6N - IDEMPOTENCIA

Unique constraints:

- CheckoutSession.
- paypalOrderId.
- paypalCaptureId.
- providerTransactionId.
- webhook eventId.

Estados:

- CREATED;
- PENDING_APPROVAL;
- APPROVED;
- CAPTURE_PENDING;
- PROVISIONING;
- COMPLETED;
- FAILED;
- EXPIRED;
- CANCELLED.

Una sesión COMPLETED no vuelve a provisionarse.

18. FASE 6O - CONCURRENCIA

Simular:

~~~text  
capture request A  
capture request B  
PayPal webhook  
browser retry  
~~~~

Resultado obligatorio:

- 1 Payment
- 1 Tenant
- 1 License
- 1 Subscription
- 1 Invoice
- 1 Membership

Nunca usar un Set de memoria como única protección.

19. FASE 6P - WEBHOOK PAYPAL

Mantener:

app/api/webhooks/paypal/route.ts
lib/services/webhook.service.ts

Exigir:

- raw body;
- verify-webhook-signature;
- webhook ID;
- eventId unique;
- persistencia PostgreSQL;
- idempotencia.

PayPal documenta la verificación de webhooks y el modelo de reentrega. ■cite■turn978776search0■

Si PostgreSQL falla al registrar el webhook:

- no marcarlo processed en memoria;

- no perder el evento;
- permitir retry/reconciliation.

20. FASE 6Q - AUTH FAIL-CLOSED

Revisar:

lib/services/auth.service.ts
lib/auth/session.ts
lib/auth/password.ts
app/api/auth/*

En producción:

PostgreSQL error -> 503/error.

Nunca:

- inMemoryUserStore;
- usuario demo;
- password demo;
- estado de usuario fabricado.

21. FASE 6R - MASTER PASSWORDS

Eliminar cualquier:

password === "..."

y cualquier contraseña maestra conocida o equivalente.

Auditar:

- auth.service.ts;
- lib/auth/*;
- app/api/auth/*;
- componentes client.

Bootstrap del primer SUPER_ADMIN, si es necesario:

scripts/bootstrap-super-admin.ts

Variables:

FENIXCMS_BOOTSTRAP_ADMIN_EMAIL
FENIXCMS_BOOTSTRAP_ADMIN_PASSWORD

El password debe:

- hacer hash;
- no imprimirse;
- no escribirse en Git;
- no funcionar como backdoor permanente.

22. FASE 6S - SUPER ADMIN FAIL-CLOSED

lib/services/super-admin.service.ts

Production:

DB responde y está vacía -> empty state.

DB falla -> 503/unavailable.

Nunca sustituir por:

INITIAL_TENANTS
INITIAL_PLANS
INITIAL_LICENSES
INITIAL_APPLICATIONS
INITIAL_THEMES
INITIAL_PLUGINS

fake metrics
fake payments.

23. FASE 6T - STORAGE FAIL-CLOSED

lib/storage/storage.service.ts

Upload:

1. validar;
2. escribir bytes;
3. calcular SHA-256;
4. persistir referencia en DB.

Si write falla:
success=false.

Si DB falla después de write:
cleanup/compensación.

Delete:
si no se confirma la eliminación -> error.

No warning + success=true.

24. FASE 6U - CHECKSUM REAL

El checksum debe calcularse sobre los bytes reales:

~~~text  
SHA-256(fileBytes)  
~~~

Guardar:

- checksumAlgorithm;
- checksum;
- sizeBytes;
- mimeType;
- storageKey.

No usar nombre + timestamp + size como sustituto de hash.

25. FASE 6V - MEDIA API FAIL-CLOSED

app/api/media/route.ts

Eliminar en producción:

tenant_demo
tenant desde query arbitraria
tenant desde body arbitrario
seed media como respuesta de error
success=true después de fallo de storage/DB.

Tenant debe venir de:

- sesión;
- trusted tenant context;
- hostname verificado.

26. FASE 6W - STORE ORDER FAIL-CLOSED

Mantener las protecciones actuales de:

- precio server-side;
- stock server-side;
- tenant server-side;

- idempotencia;
- cantidades enteras;
- estado del producto.

Pero cerrar cualquier fallback production de:

- INITIAL_PRODUCTS;
- INITIAL_TENANTS;
- FENIX10 fake coupon;
- stock en memoria.

DB failure -> error.

No demo data.

27. FASE 6X - CANCELACIÓN DE STOCK

Revisar updateOrderStatus().

No usar:

catch() => {}

en una operación donde el stock deba restaurarse.

Si la restauración falla:

- transaction failure;
- estado recuperable;
- no ocultar el error.

28. FASE 6Y - PRODUCTION AUDIT RUNTIME

Crear:

scripts/audit-production-runtime.ts

Debe escanear runtime y detectar:

- INITIAL_;
- FALLBACK_;
- tenant_demo;
- MEMORY_;
- inMemory;
- mock_;
- fake_;
- test_;
- URLs de proveedores ficticias;
- passwords hardcoded;
- valores monetarios por defecto;
- success=true después de catch.

Debe ignorar:

- tests;
- documentación;
- fixtures explícitos.

Debe fallar si el patrón aparece en runtime production.

29. FASE 6Z - SECRET AUDIT

Crear:

scripts/audit-production-secrets.ts

Detectar:

- DATABASE_URL en client;
- PAYPAL_CLIENT_SECRET en client;
- STRIPE_SECRET_KEY en client;
- hardcoded passwords;

- hardcoded access tokens;
- private keys.

No imprimir el secreto.

30. FASE 6AA - PRODUCTION ENV AUDIT

Crear:

scripts/check-production-env.ts

Production exige:

~~~text

NODE\_ENV=production

DATABASE\_URL=...

APP\_URL=https://...

PAYPAL\_CLIENT\_ID=...

PAYPAL\_CLIENT\_SECRET=...

PAYPAL\_WEBHOOK\_ID=...

PAYPAL\_BASE\_URL=https://api-m.paypal.com

~~~

Rechazar:

- localhost;
- sandbox;
- variables críticas ausentes.

31. FASE 6AB - HEALTH/READINESS

/api/health debe separar:

Liveness:

proceso Node.

Readiness:

- PostgreSQL accesible;
- PayPal config correcta;
- storage disponible/configurado.

DB down:

HTTP 503.

Nunca 200 con datos demo.

32. FASE 6AC - CI PRODUCTION GATE

Crear:

.github/workflows/production-gate.yml

Debe ejecutar:

~~~bash

npm ci

npm run lint

npm run test

npm run build

npm run audit:production

~~~

El workflow actual de PDFs no sustituye una pipeline de validación de producción.

33. FASE 6AD - PACKAGE SCRIPTS

Añadir scripts equivalentes a:

```
~~~json
{
  "audit:runtime": "tsx scripts/audit-production-runtime.ts",
  "audit:secrets": "tsx scripts/audit-production-secrets.ts",
  "check:production-env": "tsx scripts/check-production-env.ts",
  "audit:production": "npm run audit:runtime && npm run audit:secrets && npm run check:production-env"
}
~~~
```

Adaptar al package.json real.

34. FASE 6AE - TESTS FENIXCMS_6

Crear:

tests/fenixcms-fase6.test.ts

Pruebas mínimas:

P01 - DB failure Application -> no fallback.

P02 - DB failure Plan -> no fallback.

P03 - Checkout -> no Tenant.

P04 - Checkout -> no INITIAL_.*.

P05 - providerPaymentId inventado -> reject.

P06 - amount mismatch -> reject.

P07 - currency mismatch -> reject.

P08 - reference mismatch -> reject.

P09 - capture PENDING -> no provisioning.

P10 - capture DENIED -> no provisioning.

P11 - capture COMPLETED -> provisioning.

P12 - duplicate capture -> idempotent.

P13 - concurrent capture -> one result.

P14 - webhook duplicate -> one effect.

P15 - invalid webhook -> reject.

P16 - payment DB update failure -> no false COMPLETED.

P17 - subscription failure -> rollback/recoverable.

P18 - invoice failure -> rollback/recoverable.

P19 - auth DB failure -> 503.

P20 - admin DB failure -> unavailable.

P21 - empty DB -> empty state.

P22 - storage write failure -> no success.

P23 - checksum -> exact SHA-256.

P24 - runtime fallback scanner -> pass.

P25 - secret scanner -> pass.

P26 - env scanner -> pass.

35. FASE 6AF - DB VACÍA VS DB CAÍDA

Test A:

PostgreSQL funciona pero no contiene datos comerciales.

Resultado:

- [];
- empty state.

Test B:

PostgreSQL está caída.

Resultado:

- 503;
- unavailable.

Nunca sustituir B por A.

36. FASE 6AG - RECOVERY

Caso crítico:

PayPal = COMPLETED

PostgreSQL = timeout durante provisioning.

Resultado:

- no segundo cobro;
- checkout queda recuperable;
- no crear duplicados.

Crear:

reconcilePendingSaaSPayments()

que busque pagos/capturas confirmados pero provisioning incompleto y ejecute provisioning idempotente.

37. FASE 6AH - PAYPAL IDEMPOTENCY

Usar PayPal-Request-Id cuando corresponda y mantener también la idempotencia interna de PostgreSQL.

PayPal documenta PayPal-Request-Id para proteger requests contra duplicados. ■cite■turn978776search1■turn978776search8■

38. FASE 6AI - SOURCE OF TRUTH CONTRACT

SaaS:

PostgreSQL + PayPal.

Store:

PostgreSQL + provider real.

Auth:

PostgreSQL.

Media:

persistent storage + PostgreSQL.

Webhooks:

provider verified + PostgreSQL.

No existe una segunda fuente comercial en memoria.

39. FASE 6AJ - ARCHIVOS PRINCIPALES

Revisar/modificar:

lib/services/payment.service.ts

lib/services/saas-checkout.service.ts

lib/services/paypal-gateway.service.ts

lib/services/tenant.service.ts

lib/services/order.service.ts

lib/services/webhook.service.ts
lib/services/auth.service.ts
lib/services/super-admin.service.ts
lib/storage/storage.service.ts
lib/prisma.ts
lib/storeContext.tsx
prisma/schema.prisma
package.json
scripts/run-all-tests.ts

APIs:

app/api/billing/checkout/route.ts
app/api/billing/capture/route.ts
app/api/billing/verify/route.ts
app/api/tenants/provision/route.ts
app/api/webhooks/paypal/route.ts
app/api/orders/route.ts
app/api/media/route.ts
app/api/auth/login/route.ts
app/api/auth/session/route.ts

Crear si faltan:

lib/services/saas-provisioning.service.ts
scripts/audit-production-runtime.ts
scripts/audit-production-secrets.ts
scripts/check-production-env.ts
tests/fenixcms-fase6.test.ts
.github/workflows/production-gate.yml

40. FASE 6AK - PROMPT MAESTRO PARA AI STUDIO

INICIO FASE FENIXCMS_6

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

VALIDACIÓN:

ORDENADOR DEBIAN

NO DESPLEGAR:

No tocar VPS producción hasta completar esta fase.

OBJETIVO:

Cerrar definitivamente las rutas de fallback/demo del runtime de producción.

PostgreSQL + PayPal real + storage persistente deben ser la única fuente de verdad.

PRIMER ARCHIVO PRIORITARIO:

lib/services/payment.service.ts

PASO 1 - AUDITORÍA

Antes de editar, enumera por archivo:

- INITIAL_
- FALLBACK_
- tenant_demo
- MEMORY_
- inMemory
- mock_
- fake_
- ensureTenantExists
- URLs PayPal/Stripe simuladas
- passwords hardcoded
- success=true después de catch
- valores monetarios por defecto.

No consideres un dato "seguro" simplemente porque tenga la etiqueta demo.

PASO 2 - PAYMENT

Refactoriza createSaaSCheckoutSession:

- Application solo PostgreSQL.
- Plan solo PostgreSQL.
- Precio solo PostgreSQL.
- CheckoutSession persistente.
- PayPal Order real.
- approvalUrl real.
- NO Tenant.
- NO ensureTenantExists.
- NO FALLBACK_PAYMENTS en production.

PASO 3 - PROVIDER VERIFICATION

Elimina verifyWithProvider() basada en IDs que solo parecen válidos.

Para PayPal:

- Order real.
- Capture real.
- COMPLETED.
- amount exacto.
- currency exacta.
- reference exacta.
- captureId único.

PASO 4 - PROVISIONING

Crear/consolidar SaaSProvisioningService.

Una sola transacción PostgreSQL:

Payment

User

Tenant

Membership

License

Subscription

Invoice

System Domain

CheckoutSession COMPLETED

PASO 5 - DB FAILURE

Si falla una operación crítica:

- rollback;
- no false success;
- estado retryable;
- reconciliation posterior.

Nunca marcar un objeto JavaScript como COMPLETED para sustituir PostgreSQL.

PASO 6 - AUTH

Production PostgreSQL-only.

Eliminar:

- master passwords;
- user fallback;
- demo accounts como backdoor.

PASO 7 - SUPER ADMIN

DB empty = empty.

DB down = unavailable.

Nunca INITIAL_* en production.

PASO 8 - STORAGE/MEDIA

Write real.

SHA-256 real.

Failure = error.

MediaAsset solo después de storage correcto.

No tenant_demo.

PASO 9 - STORE ORDER

Mantener precio/stock server-side.
Cerrar fallbacks production.
DB failure = error.

PASO 10 - AUDIT SCRIPTS

Crear:
scripts/audit-production-runtime.ts
scripts/audit-production-secrets.ts
scripts/check-production-env.ts

PASO 11 - CI

Crear .github/workflows/production-gate.yml.

Ejecutar:
npm ci
npx prisma generate
npm run lint
npm test
npm run build
npm run audit:production

PASO 12 - TESTS

Crear tests/fenixcms-fase6.test.ts y ejecutar todas las pruebas P01-P26 del documento.

PASO 13 - INFORME

Entregar:
- archivos modificados;
- archivos creados;
- migraciones;
- cambios Payment;
- cambios provisioning;
- cambios auth;
- cambios storage;
- auditorías;
- tests;
- lint;
- build;
- audit;
- blockers restantes.

No declarar PASS si un test no se ejecutó realmente.
Distinguir MOCK, SANDBOX y LIVE.

FIN FASE FENIXCMS_6

41. VALIDACIÓN LOCAL

ORDENADOR DEBIAN

INICIO VALIDACIÓN FASE 6

```
~~~bash
cd /home/marco/fenixcms-dev/moreno
```

```
git checkout main
git pull --ff-only
```

```
node -v
npm -v
```

```
npm ci
npx prisma generate
npm run lint
npm test
npm run build
npm run audit:production
```

```
~~~
```

Si alguno de los scripts no existe, no inventar un PASS.

Primero implementar el script y volver a ejecutar.

FIN VALIDACIÓN FASE 6

42. CRITERIOS DE ACEPTACIÓN

FENIXCMS_6 solo está terminada si:

- [] PaymentService no usa INITIAL_* en production.
- [] Checkout no crea Tenant.
- [] CheckoutSession es persistente.
- [] PayPal Order es real.
- [] Approval URL proviene del proveedor.
- [] Capture es server-side.
- [] Capture se valida completamente.
- [] Payment no se marca COMPLETED en memoria.
- [] Provisioning es atómico.
- [] Provisioning es idempotente.
- [] Browser + webhook no duplican.
- [] Webhook es verificado.
- [] Webhook es idempotente.
- [] Auth es PostgreSQL-only.
- [] No hay master passwords.
- [] Super Admin no muestra datos demo.
- [] Media no usa tenant_demo.
- [] Storage falla cerrado.
- [] Checksum es SHA-256 real.
- [] Orders fallan cerrado.
- [] Audit runtime pasa.
- [] Audit secrets pasa.
- [] Audit env pasa.
- [] npm test pasa.
- [] npm run lint pasa.
- [] npm run build pasa.
- [] production-gate pasa.

43. PUERTA FINAL ANTES DEL VPS

Después de FENIXCMS_6:

1. PostgreSQL staging real.
2. PayPal Sandbox real.
3. Create Order real.
4. Approval real.
5. Capture real.
6. Webhook real.
7. Tenant creado una sola vez.
8. License creada una sola vez.
9. Subscription creada.
10. Invoice creada.
11. Login real.
12. Acceso al CMS real.
13. Retry sin duplicados.
14. Recovery después de un fallo controlado.
15. Backup verificado.
16. Variables Live revisadas.
17. Production readiness final.

Solo entonces preparar el despliegue al VPS.

44. DECISIÓN FINAL

La máxima mejora necesaria actualmente para producción no es una nueva función visual.

Es eliminar la posibilidad de que FenixCMS tenga dos realidades:

REAL:

PostgreSQL + PayPal + storage.

FICTICIA:

INITIAL_* + memoria + tenant_demo + valores por defecto + URLs simuladas + payment success en memoria.

FENIXCMS_6 debe dejar una única realidad comercial y operativa en producción.

FIN DEL DOCUMENTO FENIXCMS_6